Lab-4: GNU Debugger

Aim:

To Study the GNU Debugger (GDB) and to analyze program flow.

Theory:

Stack is a LIFO (Last In First Out) data structure. Elements are inserted and removed from the same end called the top.

- push() — inserts an element; checks for overflow (top == SIZE - 1)
- pop() — removes an element; checks for underflow (top == -1)
- Implemented using an array stack[SIZE] with a pointer top

Queue is a FIFO (First In First Out) data structure. Elements are inserted at the rear and removed from the front.

- enqueue() — inserts at rear; checks for overflow (rear == SIZE - 1)
- dequeue() — removes from front; checks for underflow (front == -1 or front > rear)
- Implemented using an array queue[SIZE] with pointers front and rear

Singly Linked List is a dynamic data structure where each node holds data and a pointer to the next node. Memory is allocated at runtime using malloc(). Insertion is done at the head; traversal uses a temp pointer until NULL.

GDB (GNU Debugger) allows step-by-step execution of C programs. Key commands used:

- b <function> — set breakpoint
- run — start program
- n — next line (step over)
- p <variable> — print variable value
- p &<variable> — print memory address of variable

Procedure:

1. Write the Stack program (stackds.c), Queue program (queue.c), and Linked List program in C.
2. Compile each with debug symbols: gcc -std=c99 -g -o test <filename>.c
3. Launch GDB: gdb ./test
4. Set breakpoints inside push() / enqueue() / insert() functions using b.
5. Run the program with run.
6. Step through execution with n and inspect variables with p.
7. Use p &variable to observe memory addresses and verify sequential allocation.
8. Record observations at each breakpoint.

Programs:

Stack:

```c
#include <stdio.h>
#define SIZE 5
int stack[SIZE];
int top = -1;

void push(int value) {
    if (top == SIZE - 1) { printf("Overflow\n"); return; }
    stack[++top] = value;
}
void pop() {
    if (top == -1) { printf("Underflow\n"); return; }
    top--;
}
int main() {
    push(10); push(20); pop();
    return 0;
}
```

Queue:

```c
#include <stdio.h>
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) { printf("Overflow\n"); return; }
    if (front == -1) front = 0;
    queue[++rear] = value;
}
void dequeue() {
    if (front == -1 || front > rear) { printf("Underflow\n"); return; }
    front++;
}
int main() {
    enqueue(1); enqueue(2); dequeue();
    return 0;
}
```

Linked List:

```c
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; };
struct Node* head = NULL;

void insert(int value) {
  struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
  newNode->data = value;
```

```
    newNode->next = head;
    head = newNode;
}
void display() {
    struct Node* temp = head;
    while (temp != NULL) { printf("%d ", temp->data); temp = temp->next; }
}
int main() {
    insert(10); insert(20); display();
    return 0;
}
```

Output after running on GDB compiler:

gdb ./stack
break main
run
next
step # go inside push()
print top
print stack
continue

```
Breakpoint 2, push (value=10) at stackds.c:8
8           if (top == SIZE - 1) {
(gdb) n
12          top++;
(gdb) n

Breakpoint 1, push (value=10) at stackds.c:13
13          stack[top] = value;   // <-- Put breakpoint here
(gdb) n
14      }
(gdb) n
main () at stackds.c:26
26              (20);
(gdb) n

Breakpoint 2, push (value=20) at stackds.c:8
8           if (top == SIZE - 1) {
(gdb) n
12          top++;
(gdb) n

Breakpoint 1, push (value=20) at stackds.c:13
13          stack[top] = value;   // <-- Put breakpoint here
(gdb) n
14      }
(gdb) p top
$1 = 1
(gdb) p stack
$2 = {10, 20, 0, 0, 0}
(gdb) p stack[0]
$3 = 10
(gdb) p stack[1]
$4 = 20
(gdb) print &stack
$5 = (int (*)[5]) 0x555555558030 <stack>
(gdb) p &stack[1]
$6 = (int *) 0x555555558034 <stack+4>
(gdb) p &stack[0]
$7 = (int *) 0x555555558030 <stack>
```

gdb ./queue
break main
run
step # enter enqueue
print front
print rear
print queue
continue

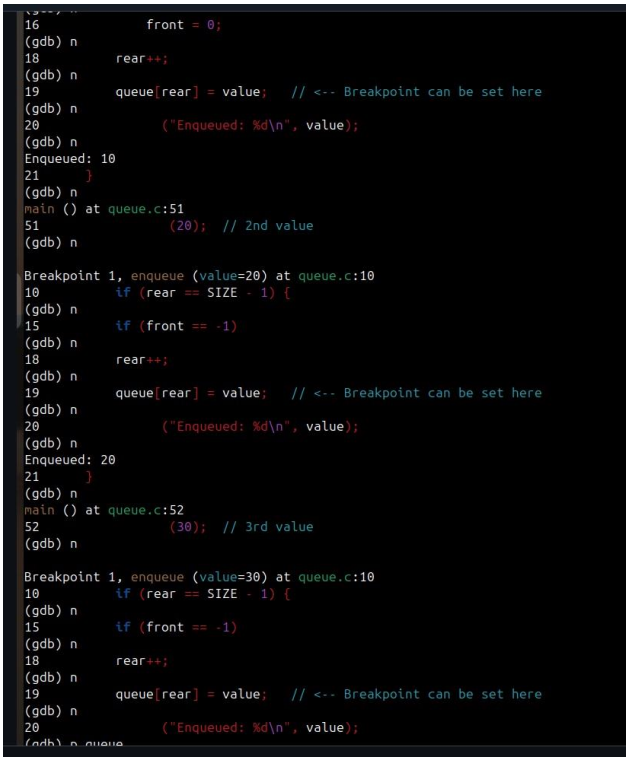
gdb ./linkedlist
break main run
step # go inside insert
print head
print *head
next
continue

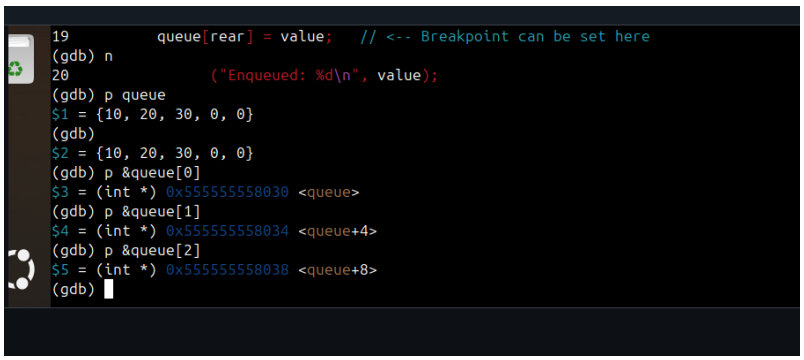
```
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ nano queue.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ gcc -std=c99 -g -o test queue.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ gdb ./test
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test...
(gdb) ^[[200~b enqueue
Undefined command: "".  Try "help".
(gdb) b enqueue
Breakpoint 1 at 0x1198: file queue.c, line 10.
(gdb) run
Starting program: /home/nitdelhi/test

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, enqueue (value=10) at queue.c:10
10          if (rear == SIZE - 1) {
(gdb) n
15          if (front == -1)
```





Conclusion:

Stack, Queue, and Singly Linked List were successfully implemented in C and debugged using GDB. The debugger confirmed correct LIFO behaviour in the Stack and FIFO behaviour in the Queue. Memory address inspection revealed that array elements are stored in contiguous memory locations, each 4 bytes apart (int size). Linked List nodes are allocated dynamically via malloc() at non-contiguous addresses. GDB breakpoints and step-through execution provided clear insight into how data structures behave at the memory level during runtime.